

A Brief Tutorial on Using Isearch

Command-line reference for indexing and searching a document collection with `Iindex` and `Isearch`.

Note. `Isearch` is included as part of `Isite`; however, `Isearch` is updated more frequently than `Isite`, so it's a good idea to use the latest standalone version rather than the one bundled with `Isite`. The latest stable version is available from `ftp.cnidr.org` in `/pub/software/Isearch/`; binaries for some systems may be available in that same directory. Both utilities print this same usage summary when run with no options.

Audience — operators & developers

Format — command reference

Revision — 1995-08-11

Iindex `[-d (X)]` Use (X) as the root name for database files.
 `[-a]` Add to existing database, instead of replacing it.
 `[-m (X)]` Load (X) megabytes of data at a time for indexing (default=1).
 `[-s (X)]` Treat (X) as a separator for multiple documents in a file.
 `[-t (X)]` Index as files of document type (X).
 `[-f (X)]` Read list of file names to be indexed from file (X).
 `[-r]` Recursively descend subdirectories.
 `[-o (X)]` Document type specific option.
 `(X) (Y) ...` Index files (X), (Y), etc.

Isearch `[-d (X)]` Search database with root name (X).
 `[-p (X)]` Present element set (X) with results.
 `[-q]` Print results and exit immediately.
 `[-and]` Perform boolean "and" on results.
 `[-byterange]` Print the byte range of each document within its file.
 `[-o (X)]` Document type specific option.
 `(X) (Y) ...` Search for words (X), (Y), etc.
 `[fieldname/]searchterm[*][:n]`
 Prefix with `fieldname/` for fielded searching.
 Append `*` for right truncation.

```
Append :n for term weighting (default=1, negative to lower rank).
```

§1

Installing Isearch

Isearch requires GNU C/C++ — both `gcc` and the `g++` library must be installed. Unpack the distribution, change into the Isearch directory, and compile:

```
make
```

After this step (or if you downloaded precompiled binaries) you'll probably want to install the executables somewhere. To install into `/usr/local/bin/` (as root):

```
make install
```

Or, to install into a custom location such as `/mylocal/bin/`:

```
make install INSTALL=/mylocal/bin
```

Isearch compiles cleanly and without modification on most UNIX platforms.

Indexing & searching a simple database (-d)

The simplest use indexes a collection of text files (e.g. *.txt) and searches the result:

```
Iindex -d /db/MYDB /data/*.txt  
Isearch -d /db/MYDB computer
```

The first command reads every *.txt file in /data/ and writes several database files beginning MYDB. into /db/. The second searches those files and displays every document that contains the word computer .

If you move or rename any of the text files you've indexed, you must re-index by running Iindex again.

Indexing & searching fielded data (-t)

Field-based searching is handled by *document types* — self-contained C++ classes that define how a class of fielded documents is indexed. At present, the only document type distributed with Isearch is `SGMLTAG`, which parses SGML-like files such as HTML, treating tag pairs as field delimiters.

For example, to index HTML files using the SGMLTAG document type:

```
Iindex -d WEBPAGES -t SGMLTAG *.html
```

The `-t` option tells Iindex to treat the files it indexes as SGML-like tagged data. To search the WEBPAGES database for the word `music` in the `title` field:

```
Isearch -d WEBPAGES title/music
```

Isearch displays every indexed document containing the word `music` between `title` and `/title` tags. Field names and search terms are case-insensitive, so the following all behave identically:

```
Isearch -d WEBPAGES title/Music  
Isearch -d WEBPAGES Title/MUSIC  
Isearch -d WEBPAGES TITLE/music
```

Document type names are also case-insensitive — `-t sgmltag` is equivalent to `-t SGMLTAG`.

Indexing files containing multiple documents (-s)

index allows many documents per file, each separated from the others by a *separator string* — a line of dashes (-----) or any other string you choose.

The `-s` option tells index what that separator is:

```
Index -d ARTICLES -s "###" *.txt
```

This treats every occurrence of `###` as a delimiter between two documents within the same `.txt` file. A file containing:

```
sample text 1  
abcde  
### sample text 2  
fghij  
### sample text 3  
klmno
```

would be interpreted as three separate documents, the second two beginning with `###`. Quotation marks around the separator aren't always required, but they're a good habit — some characters you might use (e.g. `>`) are special to the system shell.

§5

Speeding up indexing by using more memory (**-m**)

By default, `lindex` reads and indexes 1 MB of text at a time; if the text exceeds that, `lindex` saves parts of the index to disk and merges each part into the final index. With enough system memory, you can raise that block size:

```
Iindex -d MYDB -m 3 *.txt
```

This reads and indexes text in 3 MB blocks. Ideally `-m` is set larger than the total amount of text being indexed — for 1500 KB of text, use `-m 2`. In most cases there isn't enough memory to hold the entire collection at once, and `lindex` may use considerably more memory than the value given, depending on data-specific variables; `-m` specifies how much data to load, not the total memory footprint.

§6

Indexing a list of files (**-f**)

`lindex` can index a list of files you specify explicitly:

```
Iindex -d MYDB -f filelist
```

This expects `filelist` to be a plain-text listing of file names referring to the files you want indexed.

Indexing files recursively in subdirectories (**-r**)

Index can recursively descend subdirectories while searching for files to index:

```
Iindex -d MYDB -r ~/doc/
```

This indexes every file located below `~/doc/`. Note that with `-r`, the trailing file specifications are taken to be directories to search recursively — specifying a mask, for example:

```
Iindex -d WEBPAGES -t SGMLTAG -r /webdata/*.html    (incorrect!)
```

is wrong unless you actually have directory names ending in `.html`. The correct way to recursively index a web site is:

```
find /webdata/ -name "*.html" -print |  
Iindex -d WEBPAGES -t SGMLTAG -f -
```